# ARM M4 System Control Block

01 July 2023

# Content

1. Core Special registers
2. System Control Block (SCB)

# 1. Core Special registers

| Name | Type | Required privilege | Reset value | Description |
|---|---|---|---|---|
| R0-R12 | RW | Either | Unknown | General-purpose registers |
| MSP | RW | Privileged | See description | Stack Pointer |
| PSP | RW | Either | Unknown | Stack Pointer |
| LR | RW | Either | 0xFFFFFFFF | Link Register |
| PC | RW | Either | See description | Program Counter |
| PSR | RW | Privileged | 0x01000000 | Program Status Register |
| ASPR | RW | Either | Unknown | Application Program Status Register |
| IPSR | RO | Privileged | 0x00000000 | Interrupt Program Status Register |
| EPSR | RO | Privileged | 0x01000000 | Execution Program Status Register |
| PRIMASK | RW | Privileged | 0x00000000 | Priority Mask Register |
| FAULTMASK | RW | Privileged | 0x00000000 | Fault Mask Register |
| BASEPRI | RW | Privileged | 0x00000000 | Base Priority Mask Register |
| CONTROL | RW | Privileged | 0x00000000 | CONTROL register |

# 1.1. Special registers for exception or interrupt masking

## PRIMASK - Priority Mask Register

- The PRIMASK register is used to **disable all** exceptions **except** NMI and HardFault. It effectively changes the current priority level to 0 (highest programmable level).
- In C programming, you can use the functions provided in CMSIS-Core to set and clear PRIMASK:

  *void __enable_irq(); /* Clear PRIMASK */*

  *void __disable_irq(); /* Set PRIMASK */*
- In assembly language programming, you can change the value of PRIMARK register using CPS (Change Processor State) instructions:

  *CPSIE I ; Clear PRIMASK (Enable interrupts)*

  *CPSID I ; Set PRIMASK (Disable interrupts)*

# 1.2. Special registers for exception or interrupt masking

## FAULTMASK - Fault Mask Register

- The FAULTMASK is very similar to PRIMASK except that it changes the effective current priority level to -1, so that even the HardFault handler is blocked. Only the NMI exception handler can be executed when FAULTMASK is set.

- In C programming with CMSIS-compliant driver libraries, you can use the following CMSIS-Core functions to set and clear the FAULTMASK as follows:
  *void __enable_fault_irq(void); // Clear FAULTMASK*
  *void __disable_fault_irq(void); // Set FAULTMASK to disable*

- For assembly language users, you can change the current status of the FAULTMASK using CPS instructions as follows:
  *CPSIE F ; Clear FAULTMASK*
  *CPSID F ; Set FAULTMASK*

- FAULTMASK is cleared automatically upon exiting the exception handler except return from NMI handler. This characteristic provides an interesting usage for FAULTMASK: if in a lower-priority exception handler we want to trigger a higher-priority handler (except NMI), but want this higher-priority handler to start AFTER the lower-priority handler is completed, we can:
  - Set the FAULTMASK to disable all interrupts and exceptions.
  - Set the pending status of the higher-priority interrupt or exception
  - Exit the handler

  Because the pending higher-priority exception handler cannot start while the FAULTMASK is set, the higher-priority exception stays in the pending state until FAULTMASK is cleared, which happens when the lower-priority handler finishes. As a result, you can force the higher-priority handler to start after the lower-priority handler ends.

# 1.3. Special registers for exception or interrupt masking

## BASEPRI – Base Priority Register

- The BASEPRI register defines the minimum priority for exception processing. When BASEPRI is set to a nonzero value, it prevents the activation of all exceptions with the same or lower priority level as the BASEPRI value
- For example, if you want to block all exceptions with priority level equal to or lower than 0x60, you can write the value to BASEPRI:

```
__set_BASEPRI(0x60); // Disable interrupts with priority
                     // 0x60-0xFF using CMSIS-Core function
```

- For users of assembly language, the same operation can be written as:

```
MOVS R0, #0x60
MSR BASEPRI, R0 ; Disable interrupts with priority
                ; 0x60-0xFF
```

# 2. System Control Block (SCB)

| Address | Name | Type | Required privilege | Reset value | Description |
|---|---|---|---|---|---|
| 0xE000E008 | ACTLR | RW | Privileged | 0x00000000 | Auxiliary Control Register |
| 0xE000ED00 | CPUID | RO | Privileged | 0x410FC240 | CPUID Base Register |
| 0xE000ED04 | ICSR | RW | Privileged | 0x00000000 | Interrupt Control and State Register |
| 0xE000ED08 | VTOR | RW | Privileged | 0x00000000 | Vector Table Offset Register |
| 0xE000ED0C | AIRCR | RW | Privileged | 0xFA050000 | Application Interrupt and Reset Control Register |
| 0xE000ED10 | SCR | RW | Privileged | 0x00000000 | System Control Register |
| 0xE000ED14 | CCR | RW | Privileged | 0x00000200 | Configuration and Control Register |
| 0xE000ED18 | SHPR1 | RW | Privileged | 0x00000000 | System Handler Priority Register 1 |
| 0xE000ED1C | SHPR2 | RW | Privileged | 0x00000000 | System Handler Priority Register 2 |
| 0xE000ED20 | SHPR3 | RW | Privileged | 0x00000000 | System Handler Priority Register 3 |
| 0xE000ED24 | SHCRS | RW | Privileged | 0x00000000 | System Handler Control and State Register |
| 0xE000ED28 | CFSR | RW | Privileged | 0x00000000 | Configurable Fault Status Register |
| 0xE000ED28 | MMSR | RW | Privileged | 0x00 | MemManage Fault Status Register |
| 0xE000ED29 | BFSR | RW | Privileged | 0x00 | BusFault Status Register |
| 0xE000ED2A | UFSR | RW | Privileged | 0x0000 | UsageFault Status Register |


Thank you